

CUSTOMER FEATURE GUIDE

Zyvor QA Agent

Autonomous AI testing agent that reads requirements, generates tests, and validates every deploy.

An LLM-driven QA agent that continuously validates your web platform end-to-end — from requirement to running test to actionable report.

by ZyvorAI Labs

What is Zyvor QA Agent?

Zyvor QA Agent reads requirements from GitHub, generates Playwright tests, runs them after every deployment, detects regressions, and files plain-English reports — all coordinated by a LangGraph state machine. Beyond the pipeline it ships Mission Control, a live web console that runs 20+ QA capabilities on demand: end-to-end journey tests, API-contract and auth checks, real-time stream assertions, Core Web Vitals, visual sweeps, security probes, and recurring monitors. It is LLM-provider agnostic and degrades gracefully to rule-based fallbacks when no key is configured.

20+

QA actions in Mission Control

5

LLM providers supported

3

browser engines
(Chromium/Firefox/WebKit)

10

network & security probes

What's inside this guide

1 · Autonomous Pipeline	6 features
2 · Browser & Journey Testing	8 features
3 · API, Auth & Real-Time	3 features
4 · Performance & Web Quality	4 features
5 · Network & Security Probes	4 features
6 · AI Analysis & Reporting	5 features
7 · Mission Control Dashboard	6 features
8 · Integrations & Delivery	6 features
Getting started & support	→

Autonomous Pipeline

A LangGraph state machine turns requirements into running tests and back into reports — no human in the loop.



AI Test Generation

Reads product requirements and generates ready-to-run Playwright test specs that assert the described behavior.

Turns written requirements into working coverage without hand-authoring every test.



Spec-to-Test from GitHub

Pulls a markdown spec (or all specs and issues) straight from your product repo and produces tests for it.

Requirements and their tests stay in sync because both live in the same repo.



Natural-Language Test Creation

Describe a check in plain English with `zyvor-qa create` and get a generated Playwright test, optionally run on the spot.

Anyone can add a test without knowing Playwright or selectors.



Autonomous AI Browser Agent

Give a goal like 'create an Ubuntu VM with 2GB RAM' and the agent drives the real browser step by step to accomplish it.

Tests intent, not scripts — the agent figures out the clicks itself.



Self-Healing Autofix

After a failure the LLM proposes selector repairs, and can optionally patch the spec and re-run until it passes.

Brittle selectors heal themselves instead of paging an engineer.



Coverage Discovery & Expansion

Scans repo code and docs for untested routes and pages, reports the gaps, and generates tests to close them.

Surfaces the pages nobody remembered to test and fills them in.

No API key? Parsing, generation, analysis, and summaries all fall back to rule-based implementations — only natural-language creation strictly requires an LLM.

SECTION 2

Browser & Journey Testing

Drive real user journeys across browsers and devices, and catch visual regressions frame by frame.



E2E Flow Tests

Drives a multi-step journey — log in, navigate, fill a wizard, assert the outcome — as one continuous session recorded to a single video and Playwright trace.

Watch the whole user journey succeed or fail, then time-travel debug it.



Smoke Tests

Runs the hand-written smoke suite against any target with a single command and no LLM required.

A fast, dependency-free health check for every deploy.



Route Sweep

Screenshots every route at desktop and mobile, then pixel-diffs each shot against a saved baseline, masking dynamic content.

Catches unintended visual changes across the whole site at once.



Visual Regression

Compares captured screenshots against committed baselines with a configurable pixel-diff threshold.

Fails the build when the UI shifts unexpectedly, not when you meant it to.



Test Any Site

Crawls every page of any URL, generates a check per page, and runs them all — login and self-signed TLS supported.

Point it at a URL and get instant coverage with zero setup.



Visual Compare

Pixel-diffs two live URLs side by side, such as staging against production, and produces a diff image and percentage.

Prove a deploy changed nothing it wasn't supposed to.



Flaky Detection

Runs a suite N times and ranks each test by its flake rate.

Finds the unreliable tests before they erode trust in the whole suite.



Cross-Browser & Device

Runs on Chromium, Firefox, and WebKit with device profiles and 3G/offline network throttling.

Confirms the experience holds up beyond your own laptop's browser.

API, Auth & Real-Time

Validation that goes past the page — into your REST contracts, sessions, and live streams.



API Contract Tests

Validates REST endpoints against their OpenAPI schema and runs ordered multi-step API workflows with bearer or API-key auth.

Catches contract drift and broken endpoints before the UI ever sees them.



Auth & Session Tests

Logs in, saves a reusable session, and asserts logout, expiry, and negative-auth behavior.

Verifies access control works — and hands other tests a ready session to reuse.



Live-Data Assertions

Confirms WebSocket and SSE streams are actually delivering messages, covering reconnect, bearer/subprotocol/ticket auth, and live-region updates.

Proves real-time dashboards are live, not just loading.

A saved auth session can be reused by flow and real-time tests — and any target-site password is redacted from job status, history, and the live panel.

Performance & Web Quality

Grade speed, accessibility, SEO, and code coverage on a single pass.



Core Web Vitals

Measures and grades LCP, CLS, INP, FCP, and TTFB with device and network-throttle profiles.

[Know how fast real users perceive the page, with a letter grade per metric.](#)



Site Audit with A–F Grade

Runs per-page accessibility (axe-core), links, SEO, console errors, performance, and security-header checks into a pass/warn/fail matrix and overall grade.

[One report card for the whole quality picture of any page.](#)



Load Test

Fires N requests at a chosen concurrency and reports p50/p95/p99 latency and requests per second.

[A quick throughput and latency read without a separate load tool.](#)



V8 JS Coverage

Collects Chromium V8 JavaScript coverage per test run and aggregates it into the report as a percentage.

[Shows how much of your shipped JavaScript your tests actually exercise.](#)

Network & Security Probes

One-shot checks that inspect the wire, the headers, and the certificate.



TLS Certificate Check

Resolves DNS and reports the certificate issuer, expiry, protocol, and SANs.

Catches expiring or misconfigured certificates before browsers do.



Exposed-Path Probe

Probes for sensitive paths such as ``.env`` and ``.git/config``, aware of SPA fallbacks.

Flags accidental secret exposure that a functional test would miss.



Cookie & Header Inspection

Reports Secure/HttpOnly/SameSite cookie flags, full response headers, CORS policy, and compression.

Verifies the security and caching posture of every response.



Redirect & Routing Probes

Traces the full redirect chain, robots/sitemap presence, sitemap URLs, DNS records, and API status with JSON-path assertions.

Confirms crawlers, DNS, and routing behave exactly as intended.

Ten probes ship in total — redirects, headers, cookies, robots/sitemap, exposed paths, API check, sitemap URLs, DNS, CORS, and compression.

AI Analysis & Reporting

Every run ends in an explanation a human can act on and a bundle they can share.



LLM Failure Analysis

Reads traces and screenshots after a failure to propose a root cause and a fix.

[Turns a red X into a starting point instead of a mystery.](#)



Plain-English Summaries

Generates a human-readable run summary suitable for posting as a PR comment.

[Reviewers see what changed and what broke without reading raw logs.](#)



CSV / HTML / PDF Reports

Writes a downloadable report bundle for every executed job, with per-failure likely-cause hints.

[Share results in the format each audience actually wants.](#)



Video, Trace & Screenshot Artifacts

Captures journey videos, Playwright traces, and per-step screenshots, persisted under `reports/` and downloadable in bulk.

[See exactly what the agent saw when something went wrong.](#)



Test Health & Trends

Ranks worst-offender tests by fail count and flake rate and tracks a pass-rate sparkline over the last 30 runs.

[Spot the tests and trends that need attention over time.](#)

Mission Control Dashboard

A live console that runs every capability on demand and watches your cluster while it does.



Live Status Console

A self-refreshing dashboard with a glanceable verdict, stat tiles, and streamed per-test pass/fail output for the running job.

One screen tells you whether everything is green right now.



On-Demand Actions Panel

Launch any of 20+ QA capabilities from a card or the ⌘K command palette, with a stop button and live log.

Run any check without touching a terminal.



Recurring Schedules

Turns any job into a recurring monitor from 5 minutes to 6 hours, re-triggered by a background scheduler.

Smoke every 15 minutes, audit hourly, TLS daily — set and forget.



Kubernetes Pod Health

Shows per-pod phase, restarts, events, CPU/memory, and live log tails, with a restart button and namespace events.

Watch the platform under test and the agent's own pods in one place.



Authenticated & TLS-Ready

Optional password login (rate-limited, signed-cookie sessions) gates the dashboard, API, and artifacts; serve over HTTPS with a self-signed cert.

Safe to expose a console that can read pod logs.



Scriptable JSON API

The whole console is a thin client over documented JSON endpoints for jobs, schedules, runs, pods, and reports.

Automate the same actions the UI performs from your own tooling.

Integrations & Delivery

Wires into GitHub, your chat tools, your LLM of choice, and your cluster.



GitHub Integration

Reads specs and issues, runs on deploy events via an HMAC-verified webhook, and posts result summaries back as PR comments.

[QA rides along with your existing GitHub workflow.](#)



Slack, Teams & Email Alerts

Sends block-formatted Slack messages, Teams adaptive cards, and HTML email with the PDF report attached.

[The right people hear about failures where they already work.](#)



Provider-Agnostic LLM

Works with OpenAI, Anthropic, Azure OpenAI, Google, or a local Ollama model through a single configuration switch.

[Use the model and vendor you already trust — or none at all.](#)



Kubernetes & Docker Deploy

Ships manifests for Deployment, Service, Ingress, CronJob, RBAC, and PVC, plus a container image and a one-command remote deploy script.

[Run it as a scheduled in-cluster job or a standing service.](#)



CI/CD Workflows

Includes GitHub Actions for lint and unit tests, nightly and PR smoke runs, and post-deploy validation via repository dispatch.

[Coverage runs automatically on push, PR, schedule, and deploy.](#)



Rust Diff Accelerator

An optional `zyvor-diff` Rust binary replaces Pillow for faster screenshot comparison.

[Speeds up visual diffing on large baseline sets.](#)

GETTING STARTED

Your first 30 minutes

A suggested path to value on day one.

- 1 Install**
Copy ``.env.example`` to ``.env`` and run ``make install`` (needs Python 3.9+ and Node.js 20+).
- 2 Run a smoke test**
``zyvor-qa test`` runs the hand-written smoke suite against your target — no LLM key required.
- 3 Open Mission Control**
``zyvor-qa serve`` then browse to ``/dashboard`` to run any of the 20+ actions live.
- 4 Wire up GitHub**
Set ``ZYVOR_PRODUCT_REPO``, authenticate ``gh``, then ``zyvor-qa run --source github --spec docs/specs/my-feature.md``.
- 5 Add an LLM (optional)**
Set ``LLM_PROVIDER`` and the matching API key to unlock AI generation, analysis, and natural-language tests.

Support & contact

Zyvor QA Agent is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

Good to know: Many features are opt-in behind flags (regression, autofix, coverage expansion, V8 coverage, multi-browser, Rust diff) and are off by default. Without an LLM key the agent still runs but uses rule-based fallbacks for parsing, generation, analysis, and summaries; only ``zyvor-qa create`` strictly requires an LLM. API validation checks HTTP statuses and OpenAPI schemas rather than full business logic. The Mission Control dashboard reads pod logs, so it is intentionally not exposed through the ingress and should be protected with a password. Kubernetes panels require a reachable cluster; without one they show an offline state while everything else keeps working. Multi-browser and load testing are best-effort in-pod and capped to avoid resource exhaustion.

Zyvor QA Agent

An LLM-driven QA agent that continuously validates your web platform end-to-end — from requirement to running test to actionable report.

info@zyvor.dev · zyvor.dev